

Code-Layout, Readability and Reusability

Date	27 June 2026
Team ID	-
Project Name	EduGenie – AI-Powered Educational Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S. No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8-style 4-space indentation across all backend modules.
2	Proper File Structure	Files and folders are logically organized	Yes	backend/ split into middleware, routers, schemas, services, utils; frontend/ split into css, js, assets
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. current_user, access_token, gemini_response.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. qa_prompt(), generate_json(), save_history(), extract_text_from_file()
5	Code Comments	Inline and block comments explain logic	Yes	Section-banner comments (e.g. Auth Schemas) and docstrings on every router/service.
6	Modular Design	Code is split into reusable modules/functions	Yes	Routers call into services (gemini_service,

				auth_service, file_service) rather than duplicating logic.
7	Separation of Concerns	Business logic separated from routing/UI	Yes	Routers only orchestrate; Gemini prompt-building lives in utils/prompts.py; DB access lives in database.py
8	Error Handling	Errors are caught and handled gracefully	Yes	Global exception handler in main.py + try/except with HTTP Exception in each router

Reusable Components / Modules:

S. No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Authentication Service (auth_service.py)	Python, Fast API	Login, Registration, Profile APIs	High
2	Gemini Service (gemini_service.py)	Python, Google Gemini API	Q&A, Explanation, Quiz, Summarization, Roadmap	High
3	JWT Handler (jwt_handler.py)	Python, python-jose	Authentication for all protected routes	High
4	API Client (frontend/js/api.js)	JavaScript	Notes, History, Bookmarks, AI Tools, Settings	High
5	Prompt Templates (prompts.py)	Python	All AI functionalities	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Well-organized backend/frontend with modular folder structure and clear separation of concerns.
Readability	5	Consistent naming conventions, descriptive variables and functions, and clean code formatting.
Reusability	5	Services, utility modules, prompt templates, and API client are reused throughout the project.
Documentation / Comments	4	Important modules and APIs are documented; additional inline comments can further improve maintainability.
Overall Score	4.8	Production-ready architecture with modular design, reusable components, and maintainable codebase.